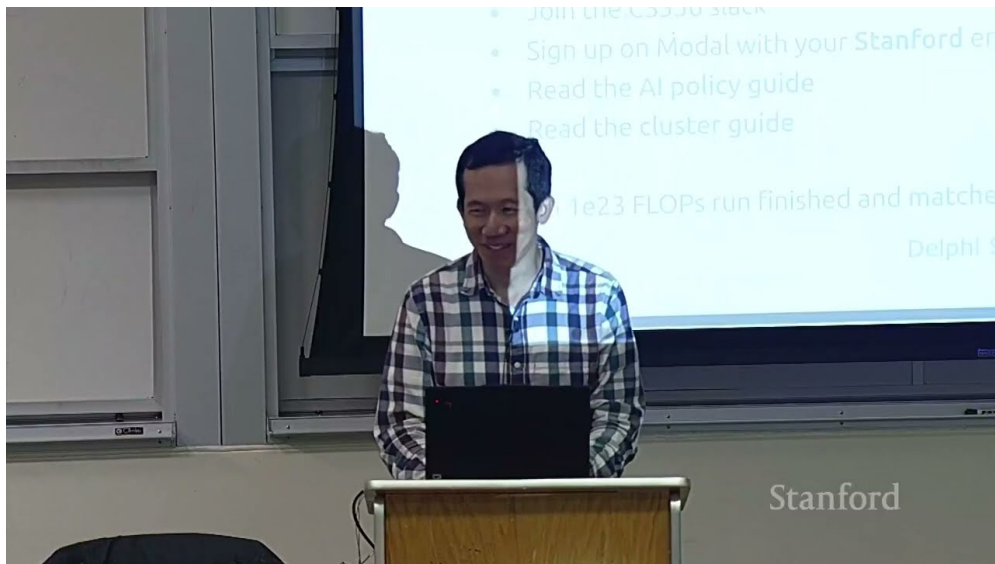


课程笔记

CS336 2026 Lecture 2: Resource Accounting

基于 Stanford CS336 Spring 2026 官方可执行讲义整理

2026 年春季



课程/频道: Stanford CS336

发布日期: Spring 2026

材料类型: 官方可执行讲义 lecture02-slides.py

课程主页: <https://cs336.stanford.edu/>

项目主页: <https://github.com/hqhq1025/ai-course-notes>

目录

1 从 tokenization 走向资源账本	2
1.1 本讲要带走的三类知识	2
1.2 本讲地图	3
1.3 本章小结	3
2 两个开场 napkin math	3
2.1 70B 模型训练时间估算	3
2.2 8 张 H100 能装多大模型	4
2.3 本章小结	4
3 Tensor 是资源账本的基本单位	5
3.1 Rank 与 Transformer 中的高维张量	5
3.2 Tensor memory 的基本公式	5
3.3 本章小结	6
4 浮点类型：范围、精度与训练稳定性	6
4.1 fp32: 传统 baseline	6
4.2 fp16: 省内存但动态范围窄	7
4.3 bf16: 深度学习训练的务实选择	7
4.4 Mixed precision: 低精度计算, 高精度状态	7
4.5 fp8 与 fp4: 更激进的低精度	8
4.6 本章小结	8
5 Device 与 GPU 数据移动	9
5.1 Device 是 tensor 状态的一部分	9
5.2 Pinned memory 与异步拷贝	9
5.3 本章小结	10
6 用 einops 给 tensor 操作命名	10
6.1 Einsum: 带维度语义的矩阵乘法	10
6.2 Reduce 与 rearrange	10
6.3 本章小结	11
7 FLOPs、FLOP/s 与 MFU	11
7.1 矩阵乘法 FLOPs	11
7.2 Model FLOPs Utilization	12
7.3 本章小结	12
8 Arithmetic intensity 与 roofline	12
8.1 两个 intensity	13
8.2 常见操作的 intensity	14
8.3 Roofline 图	15
8.4 本章小结	15

9 训练中的 forward、backward 与 6BN	16
9.1 一个两层线性网络	16
9.2 Backward 为什么约等于 forward 的两倍	16
9.3 这个近似的边界	17
9.4 本章小结	17
10 Optimizer 与训练显存账本	18
10.1 从 AdaGrad 到 Adam	18
10.2 训练内存的四类对象	18
10.3 训练循环	19
10.4 本章小结	19
11 Gradient accumulation: 用时间换 batch memory	20
11.1 显存收益	20
11.2 数学等价与工程差异	20
11.3 本章小结	21
12 Activation checkpointing: 用重计算换 activation memory	21
12.1 基本思想	21
12.2 PyTorch 里的写法	22
12.3 本章小结	22
13 把 Lecture 2 的账本串起来	22
13.1 常见诊断路径	23
13.2 本章小结	23
14 拓展阅读	23
14.1 本章小结	23
15 总结与延伸	24
15.1 核心要点	24
15.2 与后续课程的连接	24
15.3 开放问题	24

1 从 tokenization 走向资源账本

Lecture 1 解释了 CS336 的主线：在固定资源下训练最好的语言模型。Lecture 2 立刻进入这句话里最容易被忽略的一半：什么叫 fixed resources? 本讲的主题是 resource accounting, 也就是把训练系统里的计算、内存和带宽变成可以手算、可以验证、可以优化的账本。

开场的行政提醒包括加入 CS336 Slack、用 Stanford 邮箱注册 Modal、阅读 AI policy 和 cluster guide。教学上更重要的是：Marin 的 10^{23} FLOPs run 完成并匹配 forecast。这个例子把 Lecture 1 的 scaling-law 视角接到 Lecture 2 的系统账本上：如果你不能算清 FLOPs 和有效吞吐，就无法判断一个 forecast 是否可信。

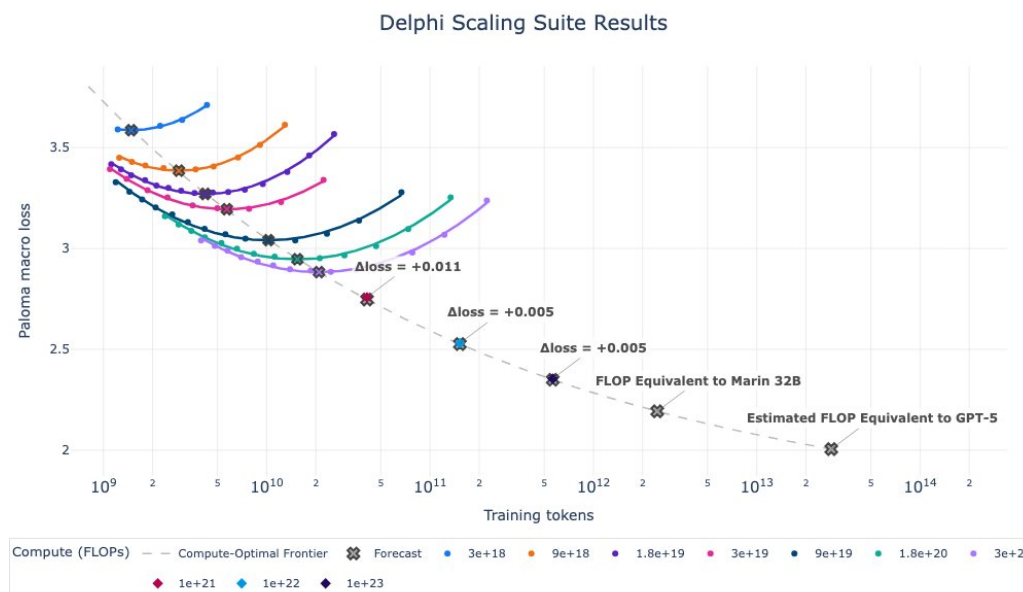


图 1: Marin 10^{23} FLOPs run 匹配 forecast, 说明资源估算不是纸上练习, 而是大规模训练决策的一部分。

来源: 官方可执行讲义 `lecture02-slides.py` 引用的公开 X/Twitter 图片: <https://pbs.twimg.com/media/HE1P1HmaUAAjLXF>。

1.1 本讲要带走的三类知识

讲义仍沿用 Lecture 1 的三分法:

- **Mechanics**: PyTorch tensor、autograd、optimizer、device、dtype 的语义。
- **Mindset**: 任何模型训练前都先做 resource accounting。
- **Intuitions**: 理解资源花在哪里。本讲没有神秘的 ML magic, 重点是算清楚。

Resource accounting 的目标

Resource accounting 不是为了把每个 kernel 的成本精确到最后一位, 而是为了形成可操作判断: 一个操作是 compute-bound 还是 memory-bound? 一个模型是参数状态装不下, 还是激活值装不下? 一次训练 step 的 FLOPs 是否和 wall-clock 吞吐匹配?

1.2 本讲地图

本讲按以下路径展开：

1. 从 tensor 开始，理解 shape、rank、dtype、device 和 memory。
2. 用 einops 给 tensor 维度命名，降低高维操作的心智负担。
3. 计算矩阵乘法 and 常见操作的 FLOPs。
4. 用 arithmetic intensity 和 roofline 判断 compute-bound / memory-bound。
5. 分解训练中的 forward、backward、optimizer state、activation memory。
6. 用 gradient accumulation 和 activation checkpointing 在显存与计算之间做交换。

1.3 本章小结

Lecture 2 把“效率”变成账本。后续 systems、parallelism、inference、scaling laws 都依赖这里的基本能力：看到一段代码或一个训练配置，先估算它要多少显存、多少 FLOPs、可能被哪个硬件资源卡住。

2 两个开场 napkin math

讲义先用两个问题训练直觉：

1. 训练一个 70B 参数模型，在 15T tokens 上，用 1024 张 H100，大概需要多久？
2. 只看参数、梯度和 AdamW optimizer state，8 张 80GB H100 上最多能装多大的模型？

2.1 70B 模型训练时间估算

训练 dense Transformer 的常用近似是：

$$C \approx 6ND$$

- C ：总训练计算量，单位为 FLOPs。
- N ：模型参数量。
- D ：训练 token 数。
- 系数 6：粗略来自 forward 的 $2ND$ 和 backward 的 $4ND$ 。

对 $N = 70 \times 10^9$ 、 $D = 15 \times 10^{12}$ ：

$$C = 6 \times 70 \times 10^9 \times 15 \times 10^{12} = 6.3 \times 10^{24} \text{ FLOPs}$$

讲义设 H100 dense bf16 峰值约为 1979/2 TFLOP/s，即 9.895×10^{14} FLOP/s；MFU 假设为 0.5；GPU 数为 1024。每天有效 FLOPs 为：

$$9.895 \times 10^{14} \times 0.5 \times 1024 \times 86400 \approx 4.38 \times 10^{22} \text{ FLOPs/day}$$

训练天数约为：

$$\frac{6.3 \times 10^{24}}{4.38 \times 10^{22}} \approx 144 \text{ days}$$

Listing 1: 官方讲义中的训练时间估算逻辑

```
1 total_flops = 6 * 70e9 * 15e12
2 h100_flop_per_sec = 1979e12 / 2
3 mfu = 0.5
4 flops_per_day = h100_flop_per_sec * mfu * 1024 * 60 * 60 * 24
5 days = total_flops / flops_per_day
```

MFU 是估算里最敏感的旋钮

如果 MFU 从 0.5 降到 0.35，同样的总 FLOPs 会把训练时间从约 144 天推到约 206 天。大规模训练里，“只有 10% 的吞吐差异”可能意味着几周排期和巨额成本。

2.2 8 张 H100 能装多大模型

第二个问题只看静态状态，不算 activations。讲义采用的粗略账本是：

$$\text{bytes per parameter} = 2 + 2 + 4 + 4 = 12$$

- 第一个 2：bf16 参数。
- 第二个 2：bf16 梯度。
- 4 + 4：AdamW 的两个 fp32 optimizer states，一阶矩和二阶矩。

8 张 80GB H100 的总显存近似为 640×10^9 bytes，因此参数上界为：

$$\frac{640 \times 10^9}{12} \approx 53.3 \times 10^9$$

也就是约 53B 参数。这仍然是乐观上界，因为它没有算 activations、fragmentation、通信 buffer、CUDA workspace、checkpointing buffer 等。

12 bytes 还是 16 bytes ?

不同账本会得到不同 bytes/parameter。Lecture 2 的开场估算使用 $2 + 2 + 4 + 4 = 12$ ，没有显式加入 fp32 master weights。有些混合精度实现会额外保存 fp32 master copy，于是变成 $2 + 2 + 4 + 4 + 4 = 16$ 。做工程估算时必须写清楚自己的假设。

2.3 本章小结

两个 napkin math 说明 resource accounting 的风格：不追求一开始就精确，而是先把数量级算清楚。70B 训练时间主要由 $6ND$ 、GPU 峰值、MFU 和 GPU 数决定；模型最大规模则先由每参数静态字节数和总显存决定。

3 Tensor 是资源账本的基本单位

在 PyTorch 里，数据、参数、梯度、optimizer state、activations 最终都以 tensor 的形式存在。理解 tensor 的 rank、shape、dtype 和 device，是理解训练资源的前提。

3.1 Rank 与 Transformer 中的高维张量

讲义从最简单的 tensor 形状开始：

Listing 2: Tensor rank 的最小例子

```
1 x = torch.zeros(4)           # rank 1 tensor: vector
2 x = torch.zeros(4, 8)        # rank 2 tensor: matrix
3 x = torch.zeros(4, 8, 2)     # rank 3 tensor
```

Transformer 中经常看到 rank 4 tensor，例如：

$$x \in \mathbb{R}^{B \times S \times H \times D}$$

- B : batch size。
- S : sequence length。
- H : number of heads。
- D : hidden dimension per head。

许多 shape bug 的根源不是数学错误，而是维度语义没有写出来。Lecture 2 后面引入 einops，就是为了解决这个心智负担。

3.2 Tensor memory 的基本公式

一个 tensor 的内存只由两个量决定：

$$\text{memory bytes} = \text{numel()} \times \text{element_size()}$$

- `numel()`: 元素个数。
- `element_size()`: 每个元素占多少 bytes，由 dtype 决定。

Listing 3: 官方讲义中的 tensor memory helper

```

1 def get_memory_usage(x: torch.Tensor):
2     return x.numel() * x.element_size()
3
4 x = torch.zeros(4, 8)
5 assert x.dtype == torch.float32
6 assert x.numel() == 4 * 8
7 assert x.element_size() == 4
8 assert get_memory_usage(x) == 4 * 8 * 4 # 128 bytes

```

同一个公式也能解释 GPT-3 feedforward layer 中一个矩阵为什么会达到 GB 级：如果矩阵形状是 $(12288 \times 4, 12288)$ ，float32 每元素 4 bytes，那么它大约是 2.3 GB。

参数量不是显存账本的全部

训练时至少要同时考虑参数、梯度、optimizer state 和 activations。参数是静态成本；activation memory 随 batch size、sequence length 和层数变化；optimizer state 取决于优化器；gradient buffer 则通常和参数同形状。

3.3 本章小结

Tensor 是 CS336 resource accounting 的基本记账单位。只要知道 shape 和 dtype，就能估算内存；只要知道 tensor operation，就能估算 FLOPs。后续所有训练系统问题都会回到这两件事。

4 浮点类型：范围、精度与训练稳定性

讲义依次介绍 fp32、fp16、bf16、fp8 和 fp4。它们的区别不是“精度高低”这么简单，而是内存、吞吐、动态范围和数值稳定性的共同取舍。

4.1 fp32: 传统 baseline

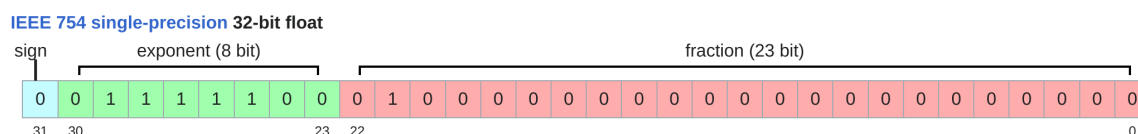


图 2: fp32 的位分配：1 位符号、8 位指数、23 位尾数。

来源：Stanford CS336 Spring 2026 官方可执行讲义 `lecture02-slides.py`，图片 `images/fp32.png`。

fp32 每个数占 4 bytes，传统科学计算中常作为 baseline。深度学习早期也常用 fp32 训练，因为它动态范围大、尾数精度高、数值行为直观。但在大模型训练中，fp32 的问题很明显：显存翻倍，Tensor Core 吞吐也远低于低精度格式。

4.2 fp16: 省内存但动态范围窄

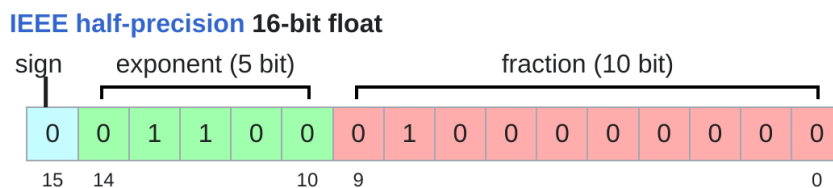


图 3: fp16 的位分配: 指数位较少, 动态范围明显小于 fp32。

来源: Stanford CS336 Spring 2026 官方可执行讲义 `lecture02-slides.py`, 图片 `images/fp16.png`。

fp16 每个数占 2 bytes, 可以把参数、激活和梯度内存减半, 也能使用更快的低精度矩阵乘法。但它的指数位只有 5 位, 小数值容易 underflow。讲义给出的例子是:

Listing 4: fp16 underflow 示例

```
1 x = torch.tensor([1e-8], dtype=torch.float16)
2 assert x == 0 # Underflow
```

训练中如果梯度或更新量落到 fp16 表示范围之外, 就可能导致不稳定。早期 mixed precision 训练常用 loss scaling 来缓解这个问题。

4.3 bf16: 深度学习训练的务实选择

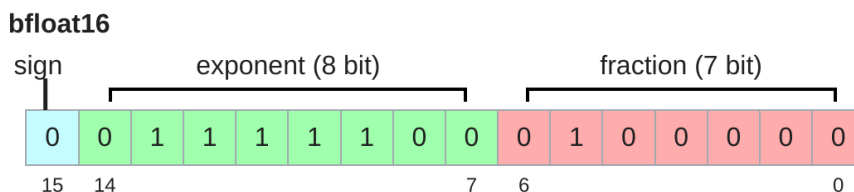


图 4: bf16 保留 fp32 的 8 位指数, 但减少尾数位。

来源: Stanford CS336 Spring 2026 官方可执行讲义 `lecture02-slides.py`, 图片 `images/bf16.png`。

bf16 也是 2 bytes, 但它有和 fp32 相同的指数范围, 代价是尾数精度更低。深度学习训练通常更需要动态范围而不是极高尾数精度, 因此 A100/H100 等硬件上 bf16 成为主流选择。

fp16 和 bf16 都是 2 bytes, 但不能混为一谈

fp16 的尾数更长、指数更短; bf16 的尾数更短、指数更长。训练中, 梯度尺度可能跨越多个数量级, 因此 bf16 的动态范围优势通常比 fp16 的尾数优势更重要。

4.4 Mixed precision: 低精度计算, 高精度状态

混合精度训练的典型策略是:

- 参数、activations、gradients 使用 bf16。

- optimizer states 使用 fp32。
- 对 matmul 等安全操作自动 cast 到低精度。
- 对 exp、softmax、某些 normalization 等敏感操作保留更高精度。

PyTorch 提供 AMP:

Listing 5: PyTorch AMP 的基本形式

```
1 with torch.amp.autocast("cuda", dtype=torch.bfloat16):
2     x = torch.zeros(4, 8)
```

为什么 optimizer state 常用 fp32

Optimizer state 会跨很多 step 累积信息，例如 Adam 的一阶矩和二阶矩。即使前向/反向可以用 bf16，长期累积状态仍需要更稳定的数值表示，否则微小更新会在多步训练中逐渐损失。

4.5 fp8 与 fp4: 更激进的低精度

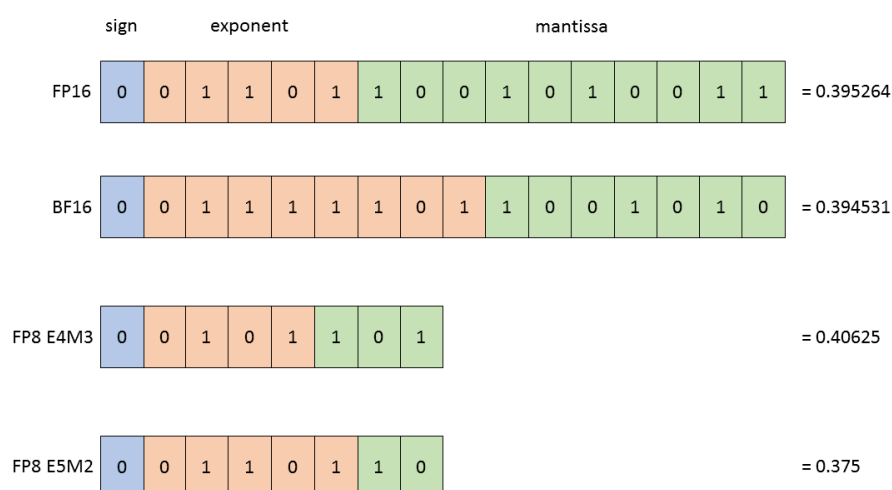


图 5: NVIDIA Transformer Engine 中的 FP8 格式示意。

来源: 官方讲义引用 NVIDIA Transformer Engine 文档图片: [fp8_formats.png](#)。

H100 支持 FP8 的 E4M3 和 E5M2 两种变体。Lecture 2 还提到 NVIDIA 2025 年的 NVFP4, 每个 value 只有 4 bits, 并配合 block scale factor 扩展动态范围。这些格式体现了一个方向: 训练和推理都在越来越精细地管理数值表示。但越低精度, 越需要硬件、kernel、scaling rule 和训练 recipe 的共同支持。

4.6 本章小结

dtype 是 resource accounting 的第一层旋钮: 它决定内存字节数, 也决定硬件峰值吞吐和数值稳定性。fp32 稳但贵, fp16 快但范围窄, bf16 是当前训练的常用折中, fp8/fp4 则代表更激进的前沿优化。

5 Device 与 GPU 数据移动

默认情况下, PyTorch tensor 在 CPU memory 中。要利用 GPU 并行性, tensor 必须移动到 GPU memory, 或者一开始就在 GPU 上创建。

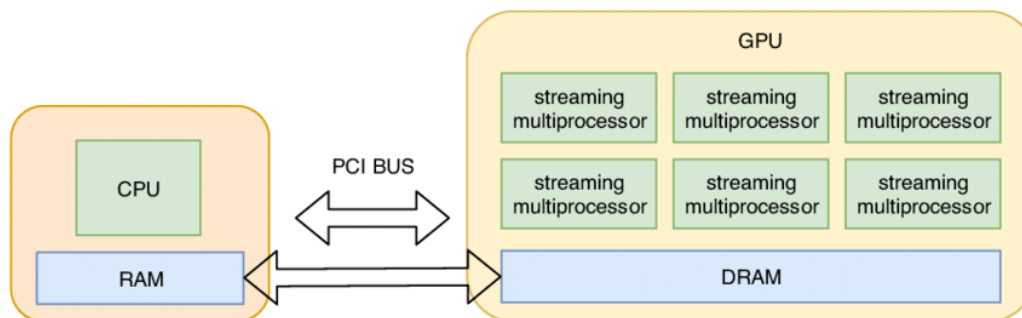


图 6: CPU 与 GPU 是不同设备, tensor 需要显式移动才能在 GPU 上计算。

来源: Stanford CS336 Spring 2026 官方可执行讲义 `lecture02-slides.py`, 图片 `images/cpu-gpu.png`。

5.1 Device 是 tensor 状态的一部分

Listing 6: CPU tensor 与 GPU tensor 的创建方式

```
1 x = torch.zeros(32, 32)
2 assert x.device == torch.device("cpu")
3
4 device = cuda_if_available()
5 x = x.to(device)
6
7 with torch.device(device):
8     x = torch.zeros(32, 32)
9     assert x.device == device
```

很多训练错误来自 device mismatch: 模型参数在 GPU, batch 还在 CPU; 或者部分 tensor 在 `cuda:0`, 另一部分在 `cuda:1`。Resource accounting 不只是纸面数字, 也包括数据是否在正确位置。

数据移动不是免费的

CPU-GPU 拷贝通过 PCIe 或 NVLink, 带宽远低于 GPU 内部 HBM 和 SM 之间的数据通路。如果训练 loop 每一步都做不必要的数据搬运, GPU 可能在等待数据, 而不是做矩阵乘法。

5.2 Pinned memory 与异步拷贝

虽然本讲脚本没有展开 DataLoader, 但 device accounting 自然引出 pinned memory。把 CPU batch 放在 pinned memory 中, 再用 `non_blocking=True` 拷贝, 可以让数据传输与 GPU 计算更好重叠。

GPU 利用率低不一定是模型太小

如果 GPU utilization 很低, 可能原因包括: CPU 数据预处理慢、CPU-GPU 拷贝阻塞、batch 太小、kernel launch 过多、通信等待或 I/O checkpoint 阻塞。资源账本必须覆盖整条训练路径。

5.3 本章小结

Device 决定 tensor 在哪里, 数据移动决定训练 pipeline 是否能持续喂饱 GPU。理解 GPU 不是“自动加速器”, 而是有独立 memory 和 bandwidth 的设备, 是后续 systems 优化的起点。

6 用 einops 给 tensor 操作命名

Tensor shape 一复杂, 传统 PyTorch 的 `transpose(-2, -1)`、`view`、`reshape` 很容易变成猜谜。Lecture 2 用 einops 展示一个更清楚的写法: 把维度名字写进操作。

6.1 Einsum: 带维度语义的矩阵乘法

Listing 7: 普通矩阵乘法与 einops einsum

```
1 x = torch.ones(3, 4) # seq1 hidden
2 y = torch.ones(4, 3) # hidden seq2
3
4 z = x @ y # seq1 seq2
5
6 z = einsum(x, y, "seq1 hidden, hidden seq2 -> seq1 seq2")
```

更复杂的 batch attention-like 操作:

Listing 8: 给 batch/sequence/hidden 命名, 避免 transpose 语义混乱

```
1 x = torch.ones(2, 3, 4) # batch seq1 hidden
2 y = torch.ones(2, 3, 4) # batch seq2 hidden
3
4 z = x @ y.transpose(-2, -1)
5 z = einsum(x, y, "batch seq1 hidden, batch seq2 hidden -> batch seq1 seq2")
```

维度命名是一种防错机制

高维 tensor 代码最危险的地方不是报错, 而是 shape 刚好对上但语义错了。Einops 把“哪个维度被求和、哪个维度保留”写出来, 可以减少 silent shape bug。

6.2 Reduce 与 rearrange

Reduce 用来对命名维度做 sum、mean、max、min:

Listing 9: einops reduce 示例

```

1 x = torch.ones(2, 3, 4) # batch seq hidden
2 y = reduce(x, "... hidden -> ...", "sum")

```

Rearrange 用来拆分或合并维度, 例如把 flatten 的 total hidden 拆成 heads 和 per-head hidden:

Listing 10: 把 total_hidden 拆成 heads 和 hidden1

```

1 x = rearrange(x, "... (heads hidden1) -> ... heads hidden1", heads=2)
2 x = einsum(x, w, "... hidden1, hidden1 hidden2 -> ... hidden2")
3 x = rearrange(x, "... heads hidden2 -> ... (heads hidden2)")

```

6.3 本章小结

Einops 的价值不是“换一种语法”, 而是把 tensor operation 的数学结构写进代码。对 resource accounting 来说, 命名维度也让 FLOPs 和 memory 估算更透明: 你知道哪几个维度相乘、哪几个维度求和。

7 FLOPs、FLOP/s 与 MFU

有了 tensor operation, 就可以估算计算量。Lecture 2 特别区分两个读音相同但含义不同的缩写:

- **FLOPs**: floating-point operations, 表示做了多少浮点操作。
- **FLOP/s 或 FLOPS**: 每秒浮点操作数, 表示硬件或实际运行速度。

7.1 矩阵乘法 FLOPs

对 $X \in \mathbb{R}^{B \times D}$ 、 $W \in \mathbb{R}^{D \times K}$, 输出 $Y = XW \in \mathbb{R}^{B \times K}$ 。每个输出元素需要 D 次乘法和约 D 次加法, 所以:

$$\text{FLOPs} \approx 2BDK$$

- B : 输入点数或 batch 维度。
- D : 输入维度, 也是求和维度。
- K : 输出维度。
- 系数 2: 乘法和加法。

Listing 11: 官方讲义中的 matmul FLOPs 计算

```
1 B = 16384
2 D = 32768
3 K = 8192
4 x = torch.ones(B, D, device=cuda_if_available())
5 w = torch.randn(D, K, device=cuda_if_available())
6 y = x @ w
7
8 actual_num_flops = 2 * B * D * K
9 actual_time = benchmark(lambda: x @ w)
10 actual_flop_per_sec = actual_num_flops / actual_time
```

7.2 Model FLOPs Utilization

MFU 定义为:

$$\text{MFU} = \frac{\text{actual FLOP/s}}{\text{promised FLOP/s}}$$

- actual FLOP/s: 通过计时和 FLOPs 估算得到的实际吞吐。
- promised FLOP/s: 硬件规格表给出的理论峰值, 且取决于 dtype。
- MFU 通常忽略通信和调度 overhead, 因此是一个近似指标。

讲义说 $\text{MFU} \geq 0.5$ 往往已经很不错。这背后不是工程师“不努力”, 而是实际训练有数据搬运、kernel launch、通信、非矩阵乘法、shape 不整齐、pipeline bubble 等各种损耗。

不要把峰值算力当作实际算力

H100 规格表里的 TFLOP/s 是理想条件下的硬件峰值。真实训练要看 dtype、矩阵形状、kernel 实现、batch size、通信、内存带宽和调度 overhead。资源估算时必须区分 theoretical peak 和 achieved throughput。

7.3 本章小结

FLOPs 衡量工作量, FLOP/s 衡量速度, MFU 衡量实际速度相对硬件峰值的比例。语言模型训练的很多工程优化, 本质上都是提高 achieved FLOP/s, 或者减少完成同样训练目标所需的 FLOPs。

8 Arithmetic intensity 与 roofline

仅知道 FLOPs 还不够。一个操作运行时间取决于计算速度和内存带宽。Lecture 2 用 arithmetic intensity 判断操作是 compute-bound 还是 memory-bound。

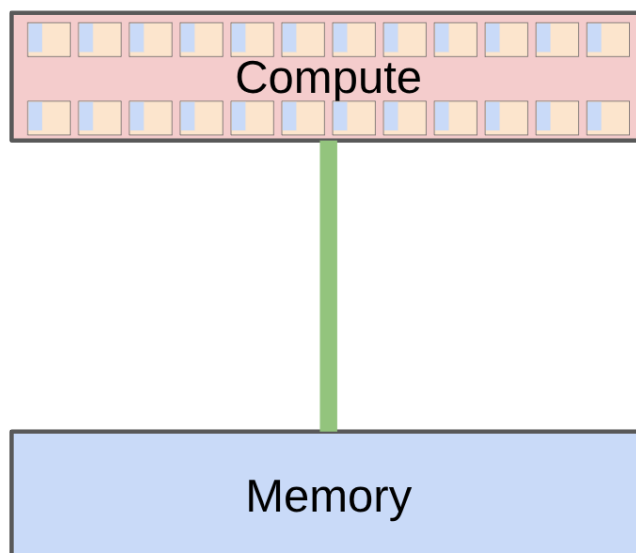


图 7: 计算操作需要从 memory 读输入，送到 accelerator 计算，再把输出写回 memory。

来源: *Stanford CS336 Spring 2026* 官方可执行讲义 `lecture02-slides.py`, 图片 `images/compute-memory.png`。

8.1 两个 intensity

硬件有 accelerator intensity:

$$\text{accelerator intensity} = \frac{\text{peak FLOP/s}}{\text{memory bandwidth bytes/s}}$$

- 对 H100, 讲义使用 dense bf16 9.895×10^{14} FLOP/s。
- H100 HBM bandwidth 约 3.35×10^{12} bytes/s。
- 因此 accelerator intensity 约为 295 FLOP/byte。

具体 workload 有 arithmetic intensity:

$$\text{arithmetic intensity} = \frac{\text{workload FLOPs}}{\text{bytes transferred}}$$

- 如果 arithmetic intensity 小于 accelerator intensity, 通常 memory-bound。
- 如果 arithmetic intensity 大于 accelerator intensity, 通常 compute-bound。

8.2 常见操作的 intensity

操作	FLOPs	Bytes	结论
ReLU over n bf16 values	n	$4n$	0.25 FLOP/byte, memory-bound
GeLU over n bf16 values	$\approx 20n$	$4n$	≈ 5 FLOP/byte, 仍 memory-bound
Dot product length n	$\approx 2n$	$\approx 4n$	≈ 0.5 , memory-bound
Matrix-vector $n \times n$	$\approx 2n^2$	$\approx 2n^2$	≈ 1 , memory-bound
Matmul $n \times n$	$\approx 2n^3$	$\approx 6n^2$	$\approx n/3$, 大矩阵 compute-bound

表 1: Lecture 2 中常见操作的 arithmetic intensity 直觉。

为什么训练喜欢大矩阵乘法

大矩阵乘法的 arithmetic intensity 随矩阵边长增长，容易超过硬件 accelerator intensity，从而充分利用 Tensor Core。逐元素操作的 intensity 基本不随规模增长，通常被内存带宽卡住。

8.3 Roofline 图

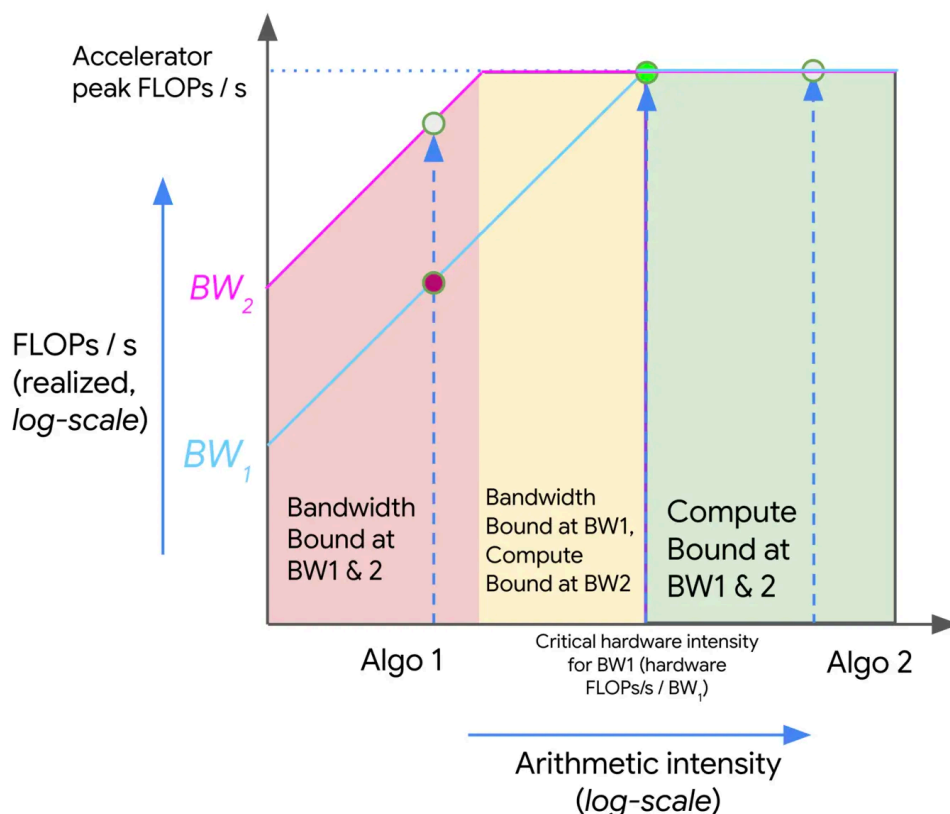


图 8: Roofline 模型把 arithmetic intensity 与可达到的 performance 连接起来。

来源: 官方讲义引用 *JAX Scaling Book roofline* 图片:

<https://jax-ml.github.io/scaling-book/assets/img/roofline-improved-1400.webp>.

Roofline 图横轴是 arithmetic intensity, 纵轴是可达到的 FLOP/s。左侧低 intensity 区域受 memory bandwidth 限制, 性能随 intensity 线性上升; 右侧高 intensity 区域受 peak compute 限制, 性能接近水平屋顶。转折点就是 accelerator intensity。

讲义把它和 MFU 连接起来:

$$\text{MFU} \approx \min \left(1, \frac{\text{arithmetic intensity}}{\text{accelerator intensity}} \right)$$

- 这是理想化表达, 忽略 kernel overhead、通信和调度。
- 它解释了为什么低 intensity 操作很难靠更强算力加速。
- 它也解释了为什么 inference 中 matrix-vector product 常 memory-bound。

8.4 本章小结

Arithmetic intensity 是连接算法和硬件的桥。FLOPs 多不一定慢, bytes 搬运多也可能是瓶颈。训练 Transformer 主要依赖大矩阵乘法, 所以更接近 compute-bound; 逐 token inference 和逐元素操作往往更接近 memory-bound。

9 训练中的 forward、backward 与 6BN

Lecture 2 接下来进入训练步骤的资源核算。核心结论是：对很多网络，单个训练 step 的 FLOPs 可以粗略写成 $6 \times \text{batch data points} \times \text{parameters}$ 。

9.1 一个两层线性网络

讲义定义一个简单 deep network，每层是线性变换加 ReLU：

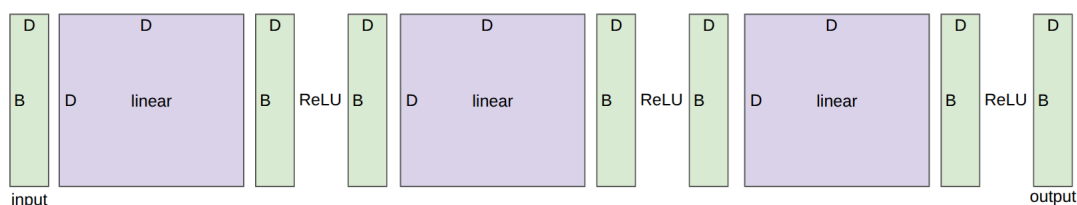


图 9: 深层网络中，每层前向产生 activations，反向需要 gradients。

来源：Stanford CS336 Spring 2026 官方可执行讲义 `lecture02-slides.py`，图片 `images/deep-network.png`。

Listing 12: 官方讲义中的简化 Block 和 DeepNetwork

```
1 class Block(nn.Module):
2     def __init__(self, dim: int):
3         super().__init__()
4         self.weight = nn.Parameter(torch.randn(dim, dim) / math.sqrt(dim))
5
6     def forward(self, x: torch.Tensor) -> torch.Tensor:
7         x = x @ self.weight
8         x = F.relu(x)
9         return x
10
11 class DeepNetwork(nn.Module):
12     def __init__(self, dim: int, num_layers: int):
13         super().__init__()
14         self.layers = nn.ModuleList([Block(dim) for i in range(num_layers)])
15
16     def forward(self, x: torch.Tensor) -> torch.Tensor:
17         for layer in self.layers:
18             x = layer(x)
19         return x
```

9.2 Backward 为什么约等于 forward 的两倍

对一层 $h_2 = h_1 W_2$ ，forward FLOPs 约为：

$$2BDD$$

- B : batch size。
- D : 输入和输出维度。

- $W_2 \in \mathbb{R}^{D \times D}$ 。

Backward 需要计算两类梯度：

- 对输入 activation 的梯度： $\frac{\partial \ell}{\partial h_1}$ 。
- 对参数的梯度： $\frac{\partial \ell}{\partial W_2}$ 。

它们各自都是一次类似规模的矩阵乘法，因此 backward 大约是 forward 的两倍。于是：

$$\text{forward} \approx 2BN, \quad \text{backward} \approx 4BN, \quad \text{training step} \approx 6BN$$

- B ：batch 中的数据点数，语言模型中常对应 token 数。
- N ：参数量。
- $6BN$ ：忽略非矩阵乘法项后的训练 step 估算。

为什么 Lecture 1 和 Lecture 2 都出现 6

Lecture 1 的 $6ND$ 是整个训练 run 的 FLOPs；Lecture 2 的 $6BN$ 是单个 training step 的 FLOPs。如果把所有 step 中的 batch token 数加起来得到总 token 数 D ，两者就是同一个近似在不同尺度上的写法。

9.3 这个近似的边界

对短上下文 dense Transformer， $6ND$ 往往是很有用的估算。但它会忽略：

- attention 的 $O(S^2)$ 项，长上下文时更明显。
- embedding 和 LM head 的词表相关成本。
- LayerNorm、softmax、activation function 等逐元素或 reduction 操作。
- MoE 的稀疏专家、routing 和 load balancing。
- 通信和 pipeline bubble。

近似不是许可你忽略细节

$6ND$ 适合快速预算，但不能替代 profiler。真正定位瓶颈时，仍要看 GPU trace、kernel 时间、memory bandwidth、communication overlap 和 shape-specific kernel efficiency。

9.4 本章小结

训练 FLOPs 的核心来自 forward 和 backward 中的大矩阵乘法。Forward 约 $2BN$ ，backward 约 $4BN$ ，合计 $6BN$ 。这个近似把模型参数量、batch token 数和训练计算量直接连起来，是后续 scaling 和 systems 分析的共同语言。

10 Optimizer 与训练显存账本

训练不只是 forward/backward, 还要更新参数。Optimizer 不仅改变更新规则, 也引入额外 state memory。

10.1 从 AdaGrad 到 Adam

讲义用 AdaGrad 作为教学优化器, 并把常见优化器关系串起来:

- Momentum = SGD + exponential averaging of gradients。
- AdaGrad = SGD + averaging by squared gradients。
- RMSProp = AdaGrad but with exponential averaging of squared gradients。
- Adam = RMSProp + momentum。

Listing 13: 教学版 AdaGrad: 每个参数保存一份平方梯度累积

```
1 class AdaGrad(torch.optim.Optimizer):
2     def __init__(self, params: Iterable[nn.Parameter], lr: float = 0.01):
3         super(AdaGrad, self).__init__(params, dict(lr=lr))
4
5     def step(self):
6         for group in self.param_groups:
7             lr = group["lr"]
8             for p in group["params"]:
9                 state = self.state[p]
10                grad = p.grad.data
11
12                g2 = state.get("g2", torch.zeros_like(grad))
13                g2 += torch.square(grad)
14                state["g2"] = g2
15
16                p.data -= lr * grad / torch.sqrt(g2 + 1e-5)
```

这个代码揭示 optimizer state 的来源: 它不是抽象概念, 而是和每个 parameter 同形状的 tensor。AdaGrad 保存一份 $\sum g^2$, Adam 保存两份 m_t 和 v_t 。

AdaGrad 的衰减问题

AdaGrad 的 g^2 是累计和, 只增不减。因此有效学习率 $\eta/\sqrt{g^2 + \epsilon}$ 会逐渐变小, 训练后期可能过早停滞。RMSProp 和 Adam 通过指数滑动平均缓解这一点。

10.2 训练内存的四类对象

对一个 toy deep network, 讲义分解:

$$M_{\text{train}} = M_{\text{params}} + M_{\text{activations}} \\ + M_{\text{gradients}} + M_{\text{optimizer state}}$$

对象	典型 dtype	随什么增长
Parameters	bf16/fp32	参数量 N
Gradients	bf16/fp32	参数量 N
Optimizer states	fp32	参数量 N ，状态份数取决于优化器
Activations	bf16/fp32	batch size、sequence length、hidden size、层数

表 2: 训练显存账本的四大类。

讲义示例中使用 bf16 参数和梯度、fp32 optimizer state:

$$\text{parameter memory} = 2N, \quad \text{gradient memory} = 2N, \quad \text{AdaGrad state} = 4N$$

Adam 则通常需要 $8N$ bytes 的两个 fp32 states。

Optimizer state 往往比参数本身更贵

如果参数是 bf16，每个参数 2 bytes；Adam 的两个 fp32 states 合计 8 bytes，是参数本身的 4 倍。只说“7B 参数模型需要 14GB”会严重低估训练显存。

10.3 训练循环

标准训练循环可以压缩成四步：取 batch、forward 计算 loss、backward 计算 gradients、optimizer step 更新参数并清梯度。

Listing 14: 官方讲义中的最小训练循环结构

```

1 for t in range(num_train_steps):
2     x, y = get_batch()
3
4     pred_y = model(x).mean()
5     loss = F.mse_loss(pred_y, y)
6
7     loss.backward()
8
9     optimizer.step()
10    optimizer.zero_grad(set_to_none=True)

```

为什么要 zero gradients

PyTorch 默认会把梯度累加到 `p.grad` 上，而不是每次 backward 自动覆盖。这是为了支持 gradient accumulation，但也意味着普通训练循环必须在 optimizer step 后清梯度，否则下一步会把旧梯度混进去。

10.4 本章小结

Optimizer 是训练资源账本的重要部分。它既决定更新规则，也决定每个参数要额外保存多少状态。一个完整训练 step 的显存必须同时考虑参数、梯度、optimizer state 和 activations，不能只

看 checkpoint 的权重大小。

11 Gradient accumulation: 用时间换 batch memory

大 batch 通常有助于训练稳定性和硬件效率, 但 activation memory 随 batch size 增长。Gradient accumulation 的思想是把一个大 batch 拆成多个 micro-batches, 逐个 forward/backward, 把梯度累加起来, 最后再更新一次。

11.1 显存收益

讲义示例中 activation memory 粗略为:

$$2 \times B \times D \times L$$

- 2: bf16 每元素 2 bytes。
- B : batch size。
- D : hidden dimension。
- L : layer 数。

如果把 batch size B 拆成 4 个 micro-batches, 每次只处理 $B/4$, 单次 activation memory 也约降为 $1/4$ 。

Listing 15: Gradient accumulation 的概念性写法

```
1 accumulation_steps = 4
2
3 for step in range(num_train_steps):
4     for micro_step in range(accumulation_steps):
5         x, y = get_micro_batch()
6         loss = loss_fn(model(x), y) / accumulation_steps
7         loss.backward()
8
9     optimizer.step()
10    optimizer.zero_grad(set_to_none=True)
```

11.2 数学等价与工程差异

如果 loss 正确缩放, gradient accumulation 的梯度等价于一个大 batch 的平均梯度。但工程上仍有差异:

- 多次 micro-batch 会增加 kernel launch 和调度 overhead。
- BatchNorm 等依赖 batch statistics 的层不完全等价; LLM 常用 LayerNorm, 问题小很多。
- 梯度通信可以延迟到 accumulation 结束, 但分布式实现需要正确设置同步策略。
- wall-clock 可能变慢, 因为用更多时间换显存。

最常见错误：忘记除以 accumulation steps

如果每个 micro-batch 的 loss 不除以累积步数，最终梯度会放大 k 倍，相当于学习率被放大 k 倍。这类 bug 可能不会立刻报错，但会让训练不稳定。

11.3 本章小结

Gradient accumulation 用更多小 step 模拟大 batch，主要节省 activation memory。它不减少总计算量，甚至可能降低吞吐，但能让显存不足的训练配置跑起来，是大模型训练中非常常见的折中。

12 Activation checkpointing：用重计算换 activation memory

训练时，forward 需要保存每层 activation，backward 才能计算梯度。对深层网络，activation memory 可能成为显存瓶颈。Activation checkpointing 又叫 gradient checkpointing 或 rematerialization，它用重计算换显存。

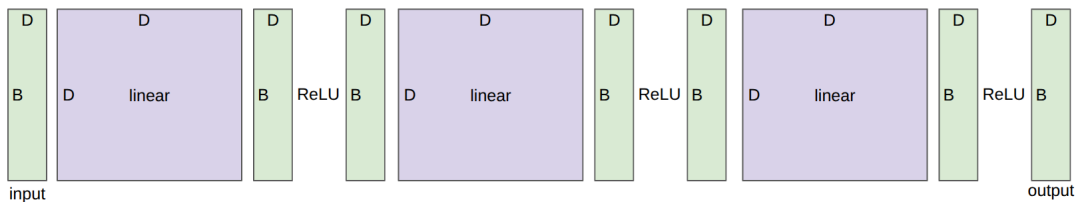


图 10: Activation checkpointing 的对象是深层网络中逐层产生的 activations。

来源：Stanford CS336 Spring 2026 官方可执行讲义 `lecture02-slides.py`，图片 `images/deep-network.png`。

12.1 基本思想

普通训练保存所有层的 activations：

$$x, h_1, h_2, h_3, \dots, h_L$$

Checkpointing 只保存一部分 checkpoints。Backward 需要中间 activation 时，从最近 checkpoint 重新 forward 计算出来。

- 保存所有 activation：activation memory $O(L)$ ，无额外重计算。
- 不保存中间 activation：memory $O(1)$ ，但可能产生 $O(L^2)$ 重计算。
- 每 $\sqrt{L}$ 层保存一次：memory $O(\sqrt{L})$ ，重计算 $O(L)$ 量级。

Activation checkpointing 的核心交易

它不是让训练“免费省内存”，而是用额外 compute 换较低 activation memory。当训练受显存限制且 GPU compute 还有余量时，这个交易非常划算；当训练已经 compute-bound 且显存足够时，它可能只会变慢。

12.2 PyTorch 里的写法

官方讲义给出 checkpointed network:

Listing 16: 用 PyTorch activation checkpointing 包住每层

```
1 class DeepNetworkCheckpointed(nn.Module):
2     def __init__(self, dim: int, num_layers: int):
3         super().__init__()
4         self.layers = nn.ModuleList([Block(dim) for i in range(num_layers)])
5
6     def forward(self, x: torch.Tensor) -> torch.Tensor:
7         for layer in self.layers:
8             x = torch.utils.checkpoint.checkpoint(layer, x)
9         return x
```

真实 Transformer 中, 常见策略是按 block 做 checkpoint, 而不是每个小算子都 checkpoint。粒度太细会导致重计算和调度 overhead 太大; 粒度太粗又节省不了足够内存。

Checkpointing 的隐藏要求

被 checkpoint 的 forward 最好是确定性的。如果其中有 dropout、随机采样或依赖全局状态的副作用, 必须确保 backward 重算时使用一致的 RNG/state, 否则梯度可能不等价。

12.3 本章小结

Activation checkpointing 是显存-计算 tradeoff 的典型例子。它减少需要保存的 activations, 让更大 batch、更长 sequence 或更深模型变得可训练; 代价是 backward 中需要重算部分 forward。

13 把 Lecture 2 的账本串起来

现在可以把本讲所有资源项放进一张统一账本:

账本项	估算公式	典型优化手段
参数内存	$\text{dtype bytes} \times N$	bf16/fp8、sharding、quantization
梯度内存	$\text{dtype bytes} \times N$	gradient sharding、zeroing strategy
Optimizer state	$\text{state bytes} \times N$	低精度 optimizer、Adafactor、ZeRO
Activation memory	$\propto BSLd$	gradient accumulation、checkpointing、sequence parallel
Training FLOPs	$\approx 6ND$	更好架构、更高 MFU、更短训练 recipe
Arithmetic intensity	FLOPs / bytes	kernel fusion、tiling、增加 reuse
Data movement	bytes / bandwidth	pinned memory、overlap、减少 CPU-GPU 往返

表 3: Lecture 2 的统一资源账本。

为什么 systems 是语言模型研究的一部分

如果一个模型在理论上更好，但需要过多 activation memory、低 arithmetic intensity、难以并行或无法稳定低精度训练，它在真实资源约束下可能并不更好。CS336 把 systems 放进主线，是因为模型质量和系统效率已经不可分离。

13.1 常见诊断路径

看到训练慢或 OOM，可以按以下顺序排查：

1. **OOM**：先分解参数、梯度、optimizer state、activations，判断是哪类内存最大。
2. **GPU utilization 低**：看数据加载、CPU-GPU 拷贝、batch size、kernel launch 和通信等待。
3. **MFU 低**：看操作是否 memory-bound、矩阵 shape 是否友好、是否有过多逐元素 kernel。
4. **训练发散**：看 dtype、loss scaling、梯度范数、optimizer state 和学习率。
5. **吞吐波动**：看 checkpoint I/O、数据源、分布式同步和动态 shape。

本讲最重要的习惯

在写训练代码之前，先写一版纸面账本；在训练跑起来之后，用 profiler、显存统计和吞吐日志验证账本。账本和观测不一致时，不要急着调参，先解释差异。

13.2 本章小结

Lecture 2 的每个概念都能放入统一账本：dtype 决定 bytes 和峰值吞吐，tensor shape 决定 memory 和 FLOPs，arithmetic intensity 决定瓶颈类型，optimizer 和 activations 决定训练显存，gradient accumulation 与 checkpointing 则在资源之间做交换。

14 拓展阅读

- Stanford CS336 Spring 2026 lecture materials: [GitHub: stanford-cs336/lectures](#)。
- PyTorch AMP documentation: [PyTorch AMP documentation](#)。
- JAX Scaling Book roofline chapter: [JAX Scaling Book: roofline](#)。
- Transformer memory accounting blog: [Transformer memory accounting](#)。
- Transformer FLOPs accounting blog: [Transformer FLOPs accounting](#)。

14.1 本章小结

拓展阅读的重点是把本讲账本用于真实系统：AMP 文档帮助理解 dtype casting，roofline 解释硬件瓶颈，Transformer memory/FLOPs 博客则把 toy network 的公式推广到真实 decoder-only Transformer。

15 总结与延伸

Lecture 2 把 CS336 的 efficiency 主线落到了最基础的工程对象上。上一讲说要在固定资源下训练最好的模型；这一讲教你先把资源数出来。没有 resource accounting, scaling law 只是漂亮曲线, systems 优化只是盲目调参, 训练 OOM 也只能靠猜。

15.1 核心要点

1. Tensor 的内存由 shape 和 dtype 决定: $\text{numel}() \times \text{element_size}()$ 。
2. dtype 同时影响内存、吞吐和稳定性; bf16 是当前训练的关键折中。
3. 矩阵乘法 FLOPs 约为 $2BDK$, 训练 dense Transformer 常用 $6ND$ 做预算。
4. MFU 衡量实际吞吐相对硬件峰值的比例, 0.5 已经很有工程含金量。
5. Arithmetic intensity 决定一个操作更可能被计算卡住还是被内存带宽卡住。
6. Optimizer state 和 activations 往往比直觉中更吃显存。
7. Gradient accumulation 用时间换 batch memory; activation checkpointing 用重计算换 activation memory。

15.2 与后续课程的连接

本讲是后续 systems 单元的地基。Kernel 优化会继续使用 arithmetic intensity 和 data movement 的语言; parallelism 会把参数、梯度、optimizer state、activations 分摊到多 GPU; inference 会重新审视 prefill 和 decode 的 compute/memory balance; scaling laws 则会把 $6ND$ 推广成训练预算和实验设计的核心坐标。

15.3 开放问题

- 当 context length 极长时, $6ND$ 近似需要怎样修正 attention 成本?
- FP8/FP4 训练能否成为通用 recipe, 还是只适合特定架构和硬件库?
- Activation checkpointing 的最优粒度能否自动选择, 而不是依赖经验?
- 对 agentic workload, 推理阶段的 memory-bound decode 是否会比训练 FLOPs 更快成为成本中心?

最终 takeaway

语言模型训练不是抽象地“跑一个模型”, 而是在 tensor、dtype、FLOPs、bandwidth、optimizer state 和 activations 之间做资源分配。Lecture 2 要训练的核心能力是: 先算账, 再优化。